

Reviewer Pack — Minimal Rank of Tropicalized Kneser Graphs Bounds Resolution...

Entry 31aed1ad0aca · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 18:00:31 UTC

Minimal Rank of Tropicalized Kneser Graphs Bounds Resolution Refutation Size

Entry ID: 31aed1ad0aca **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 18:00:31 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Kneser graphs) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{‘sentence_1’: ‘For any n -vertex Kneser graph, the Tseitin formula constructed from it requires a Resolution refutation of length at least $2^{(n/2)}$.’, ‘sentence_2’: ‘This implies that the resolution proof complexity for Kneser graphs is exponential in their size.’, ‘sentence_3’: ‘The conjecture holds for all $n \leq 40$ and can be tested within 240 seconds using pure Python.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Kneser graphs are known to have high tree-width, which makes them difficult to resolve.’, ‘sentence_2’: ‘Tropicalization of these graphs may preserve their structural complexity, making it a suitable invariant for proving lower bounds on resolution proof length.’, ‘sentence_3’: ‘This conjecture, if true, would provide a new approach to proving exponential lower bounds for Resolution refutations.’}

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cbe8d815ee85a963

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for $n = 40$ and using 30 seeds, the average resolution refutation length across all Kneser graphs exceeds $2^{(n/2)}$, otherwise it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND kneser graph AND resolution proof complexity - resolution refutation size AND tropicalized kneser graphs AND complexity theory - exponential complexity AND kneser graph AND Tseitin formula

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization - [http://arxiv.org/abs/2404.04038v1] Refutability as Recursive as Provability - [http://arxiv.org/abs/2409.18626v1] Refutation of Spectral Graph Theory Conjectures with Search Algorithms) - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/2501.18019v2] An exact closed walks series formula for the complexity of regular graphs and some related bounds - [http://arxiv.org/abs/1402.4338v2] Proof Complexity and the Kneser-Lovász Theorem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def generate_kneser_graph(n, k):
    elements = list(range(1, n + 1))
    graph = []
    for S in combinations(elements, k):
        for T in combinations(elements, k):
            if len(S & T) == 0:
                graph.append((S, T))
    return graph

def combinations(iterable, r):
    pool = tuple(iterable)
    n = len(pool)
    indices = list(range(r))
    yield tuple(pool[i] for i in indices)
    while True:
        for i in reversed(range(r)):
            if indices[i] != i + n - r:
                break
        else:
            return
        indices[i] += 1
        for j in range(i + 1, r):
```

```

        indices[j] = indices[j - 1] + 1
    yield tuple(pool[i] for i in indices)

def tseitin_formula(graph):
    literals = {}
    clauses = []
    literal_id = 0

    def get_literal(S):
        if S not in literals:
            literals[S] = literal_id
            literal_id += 1
        return literals[S]

    for (S, T) in graph:
        x = get_literal(S)
        y = get_literal(T)
        clauses.append([-x, -y])
        clauses.append([x, y])

    # Add unit clause for each vertex
    for S in literals:
        clauses.append([get_literal(S)])

    return clauses

def dpll(clauses):
    def solve(model):
        if not clauses:
            return model
        literal = next((lit for lit in range(1, max(literals.values()) + 1) if lit not in model and
        if literal is None:
            return None

        new_model = model.copy()
        new_model[literal] = True
        result = solve(new_model)
        if result is not None:
            return result

        new_model[literal] = False
        result = solve(new_model)
        if result is not None:
            return result

        return None

    return solve({})

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    graph = generate_kneser_graph(n, 2)
    clauses = tseitin_formula(graph)
    refutation_length = len(dpll(clauses)) if dpll(clauses) is not None else float('inf')
    return {

```

```

        "metric_name": "resolution_refutation_length",
        "metric_value": refutation_length,
        "instances_tested": 1,
        "conjecture_holds": refutation_length >= 2**(n/2),
        "counterexample": "" if refutation_length >= 2**(n/2) else "refutation_length < 2^(n/2)"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_refutation_length = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_refutation_length} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='refutation_length < 2^(n/2)' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62841424.py", line 108, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62841424.py", line 92, in run_trial
    graph = generate_kneser_graph(n, 2)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62841424.py", line 23, in generate_kneser_graph
    if len(S & T) == 0:
        ~^~
TypeError: unsupported operand type(s) for &: 'tuple' and 'tuple'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required computation to verify the conjecture. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12231
2	preregistration	ollama_remote	glm4:latest	0	0	5182
3	novelty	ollama_remote	glm4:latest	0	0	4625
4	novelty	ollama_remote	glm4:latest	0	0	5814
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19247
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13108
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10362
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10464
9	judge	ollama_remote	glm4:latest	0	0	8312

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 89346 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/31aed1ad0aca.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/31aed1ad0aca.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/31aed1ad0aca.tar.gz (if generated)